Boolean Algebra, Circuits, SAT, and State Models

Exhaust small cases; name what the model leaves out

COMSC 2043 ▪ Discrete Structures and Critical Thinking

Week 14, Tuesday, November 17, 2026

The question for today

PRINCIPLE: MODEL SCOPE

Week 14 question

Is this transformation equivalent under every assignment, and what can this state model not establish?

- A Boolean expression says what is true for one assignment.
- A finite-state machine says how a bounded model changes as inputs arrive.
- Both are useful only when their assumptions are visible.

PREDICT (30 s): What evidence would make these expressions are equivalent a justified claim?
What you should see: every relevant assignment is checked, not just a favorable example.

Our checking rhythm

1. Predict the row, witness, trace, or missing case.
2. Run a small, visible check or fill it by hand.
3. Explain exactly what the evidence establishes.
4. State one boundary: what was not represented or checked?

A useful rule

A fluent answer is a proposal. A complete check is evidence.

What you should see: each section ends with a row, counterexample, trace, or gap.

Words we will use

Boolean variable	A symbol that receives either true or false.
assignment	The chosen true/false value for every variable.
operator	A rule such as AND, OR, or NOT that combines or changes values.
expression	A written combination of variables and operators.
truth table	A table with one row for every assignment.
equivalent	Producing the same result for every assignment in scope.
counterexample	One in-scope case that disproves a universal claim.
circuit	A picture of gates connected to represent an expression.
gate	A circuit component that performs one Boolean operation.
state	A named condition in a changing system.
transition	A rule that changes one state to another for an input.

The tempting simplification

Supplied expression

$$F = (P \text{ OR } Q) \text{ AND } (P \text{ OR NOT } Q)$$

Confident proposed simplification

$$B = P \text{ OR } Q$$

PREDICT (30 s): If an AI says F and B are equivalent, what is the smallest check that could break the claim?

What you should see: a complete four-row truth table can find a single counterexample.

Common mistake: They match on the first row is not the same as they match on every row.

Four assignments, four rows

PRINCIPLE: EXHAUST SMALL CASES

P	Q	P OR Q	P OR NOT Q	F
False	False	False	True	False
False	True	True	False	False
True	False	True	True	True
True	True	True	True	True

PREDICT (30 s): For which row could $B = P$ OR Q disagree with F?
What you should see: the row $P = \mathit{False}, Q = \mathit{True}$ makes $B = \mathit{True}$ while $F = \mathit{False}$.

One counterexample is enough

P	Q	F	B	match?
False	False	False	False	yes
False	True	False	True	
True	False	True	True	yes
True	True	True	True	yes

Conclusion

The proposed simplification is not equivalent to F because one assignment differs.

What you should see: the exact mismatch is $P = \mathit{False}, Q = \mathit{True}$; no larger example is needed.

Prompt 01: make the claim checkable

PRINCIPLE: DEFINE BEFORE COMPUTING

PROMPT FOR CLAUDE

prompts/01_session_rules.md

```
Act as a careful checking partner for a two-variable Boolean example. Before writing code, state what an expression, a
  Boolean variable, an assignment, and equivalence mean in plain language. Use only the operators AND, OR, and NOT. Keep
  the example synthetic and self-contained. Then propose the smallest Python standard-library evaluator you would need
  for a truth table. Do not claim two expressions are equivalent until every assignment has been checked. Show one
  predicted row before showing the rest.
```

PREDICT (30 s): What must be defined before an evaluator can be trusted?

What you should see: terms, operators, assignment order, and the meaning of equivalence are visible.

Prompt 02: make every row visible

PRINCIPLE: EXHAUST SMALL CASES

PROMPT FOR CLAUDE

prompts/02_truth_table_counterexample.md

```
Write a small Python program using only the standard library. Define F = (P OR Q) AND (P OR NOT Q) and the proposed
  simplification B = P OR Q. Evaluate both expressions for all four Boolean assignments of P and Q, in the fixed order (
  False, False), (False, True), (True, False), (True, True). Print a compact table with P, Q, F, and B, then print the
  exact assignments where they differ. Do not simplify the expressions by inspection. The expected check is one mismatch,
   at P=False and Q=True.
```

PREDICT (30 s): Which row should reveal the false simplification?

What you should see: four rows and exactly one mismatch: $P = \mathit{False}, Q = \mathit{True}$.

A correct simplification from the activity

The Tuesday source activity supplies:

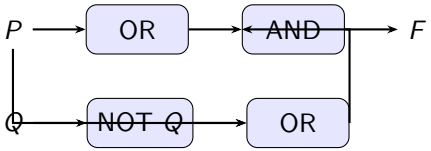
$$E = (P \text{ AND } Q) \text{ OR } (P \text{ AND NOT } Q)$$

and proposes $S = P$.

- Complete the same four rows.
- E and S agree on every row.
- A correct simplification is earned by the table, not by familiarity.

What you should see: the source example and the deliberately false example use the same checking method.

From expression to circuit



A circuit is a representation: each gate computes one operation and each wire carries a value.

Check: a circuit that omits a wire may implement a different expression.

PREDICT (30 s): If one wire is missing, should we trust the same truth table?
What you should see: the picture must be translated back into an expression before checking it.

Equivalence is a universal statement

Equivalent

Two expressions are equivalent when they return the same value for every assignment in the declared scope.

- Every is why the four-row table matters.
- One mismatch changes an equivalence claim to not equivalent.
- A matching sample is evidence for those rows, not a universal proof.

What you should see: scope = variables P, Q and operators AND, OR, NOT in these examples.

Satisfiable is not always true

satisfiable	At least one assignment makes an expression true.
witness	A displayed assignment that proves satisfiability.
tautology	An expression true for every assignment.
unsatisfiable	No assignment makes an expression true.

Example

P is satisfiable: $P = \mathit{True}$ is a witness. But P is not a tautology: $P = \mathit{False}$ is a counterexample.

PREDICT (30 s): Does one satisfying assignment establish true for every assignment?
What you should see: No. A witness proves at least one, while a tautology needs every row.

Three small classifications

PRINCIPLE: CHECK THE WORDING

Expression	Result	Evidence
P	satisfiable, not a tautology	witness $P = \mathit{True}$; counterexample $P = \mathit{False}$
P OR NOT P	tautology	both assignments are true
P AND NOT P	unsatisfiable	no assignment makes both sides true

Common mistake: I found one true row cannot be followed by therefore it is always true.

What you should see: the claim word determines the required evidence.

Prompt 03a: return counterexamples

PRINCIPLE: TEST THE UNIVERSAL CLAIM

PROMPT FOR CLAUDE

prompts/03_equivalence_sat.md

```
Extend a small standard-library Boolean evaluator with an equivalence checker. It must enumerate every assignment for the
   variables that occur and return a list of counterexamples, not just True or False. Check that E = (P AND Q) OR (P AND
  NOT Q) is equivalent to S = P, and check that F = (P OR Q) AND (P OR NOT Q) is not equivalent to B = P OR Q. Print the
  counterexample for the second pair exactly as a mapping from variable names to Boolean values. State why an empty
  counterexample list supports equivalence only for the variables and operators actually supplied.
```

PREDICT (30 s): Which pair should have an empty counterexample list?

What you should see: E versus S is equivalent; F versus B returns one exact mapping.

Prompt 03b: witness versus tautology

PRINCIPLE: DO NOT OVERCLAIM

PROMPT FOR CLAUDE

prompts/03_equivalence_sat.md

```
Using the same evaluator, write three separate checks. First, return one satisfying witness for P, meaning one assignment
    that makes P true. Second, decide whether P is a tautology, meaning true for every assignment, and if it is not, print
    one counterexample. Third, classify P OR NOT P and P AND NOT P as tautology or unsatisfiable. Explain in one sentence
  why a satisfying witness for P does not prove that P is always true. Do not use the words equivalent or always without
  showing the assignments that justify them.
```

PREDICT (30 s): What is the difference between finding one witness and checking every row?
What you should see: the output labels satisfiability, tautology, and unsatisfiability separately.

Truth tables and circuits agree when the representation agrees

PRINCIPLE: CHECK THE REPRESENTATION

- The expression is the rule.
- The circuit is a picture of that rule.
- The table is the exhaustive behavior of that rule over its assignments.

Visible check

Pick one row, evaluate the expression, follow the same inputs through the gates, and compare the outputs.

PREDICT (30 s): What could make a circuit and expression disagree even if both programs run?
What you should see: an omitted wire, wrong gate, or unstated input can change the model.

Now change the question: how does it move?

PRINCIPLE: FROM VALUES TO STATES

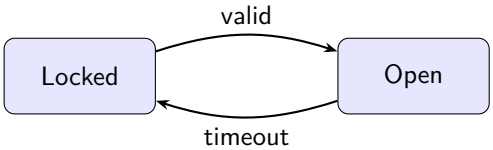
A **finite-state machine** is a bounded model with named states and input-triggered transitions.

input	An event supplied to the machine.
start state	The state where a trace begins.
transition	The declared next-state rule for a state and input.
trace	The ordered states visited as inputs are processed.
model limit	A relevant fact the model does not represent or establish.

PREDICT (30 s): What must be named before another person can reproduce a trace?

What you should see: states, input alphabet, start state, transitions, and input sequence.

Synthetic badge model



Declared inputs: valid, timeout.

Start: Locked.

Classroom rule: other inputs leave the state unchanged.

PREDICT (30 s): Starting at Locked, what states follow valid, timeout, valid?

What you should see: Locked $\rightarrow$ Open $\rightarrow$ Locked $\rightarrow$ Open.

Common mistake: An input is not a state; it is the event used to choose a transition.

Trace it one input at a time

step	input	state after input
start	—	Locked
1	valid	Open
2	timeout	Locked
3	valid	Open

Trace

```
Locked -> Open -> Locked -> Open
```

What you should see: the trace includes the start state so another person can reproduce it.

Prompt 04: run the complete model

PRINCIPLE: REPRODUCIBLE TRACE

PROMPT FOR CLAUDE

prompts/04_fsm_trace.md

```
Implement a tiny finite-state-machine runner in Python with no third-party packages. The machine has states Locked and
  Open, inputs valid and timeout, start state Locked, and these transitions: (Locked, valid) -> Open and (Open, timeout)
  -> Locked. For this classroom trace, every other input leaves the state unchanged. Run the input sequence valid,
  timeout, valid and print the start state plus each resulting state. Also print a visible error for an input that is not
   in the declared alphabet. The expected state sequence is Locked, Open, Locked, Open. Do not invent authorization facts
   or physical-reader behavior.
```

PREDICT (30 s): What should the final state be after the three inputs?

What you should see: the exact four-state sequence is Locked, Open, Locked, Open.

A model can be incomplete

PRINCIPLE: FIND WHAT WAS FORGOTTEN

Declared states: Locked, Open
Inputs: valid, timeout

Present	Locked + valid → Open
Missing	Open + timeout → Locked

Do not fill it silently

An implicit self-loop is not the same as a declared transition. List the state/input pairs and compare them with the table.

PREDICT (30 s): Which pair must a validator report?
What you should see: exactly (*Open*, *timeout*) is absent from the intentionally incomplete model.

Prompt 05: name the missing transition

PRINCIPLE: VALIDATE THE MODEL

PROMPT FOR CLAUDE

prompts/05_model_limit.md

```
Write a validator for a finite-state machine whose declared states are Locked and Open and whose input alphabet is valid
  and timeout. The intentionally incomplete transition table contains only (Locked, valid) -> Open. List every missing (
  state, input) pair in deterministic order. The validator must identify exactly (Open, timeout) as the missing
  transition needed by the badge trace. Do not assume an implicit self-loop counts as a declared transition. Explain in
  one sentence what this validator can establish and what it cannot establish about a real access system.
```

PREDICT (30 s): What should appear in the missing-transition list?
What you should see: exactly (*Open*, *timeout*), with no automatic self-loop.

What the badge model cannot establish

PRINCIPLE: MODEL LIMIT

The two-state model can establish only claims about its declared states, inputs, transitions, start state, and trace.

- It does not establish whether a person was authorized.
- It does not establish whether a physical reader worked.
- It does not represent denied, expired, or tampered conditions.

Strong conclusion

The trace is correct for the declared model. It is not a proof that a real access system is safe.

What you should see: a limitation is a precise omitted condition, not a vague disclaimer.

The three AI failure modes

Failure	Check	Visible evidence
False simplification	all truth-table rows	one counterexample row
Witness called universal proof	separate SAT/tautology tests	witness plus remaining counterexample
Omitted state or transition	declared Cartesian product	named missing pair

PREDICT

(30 s): Which check would you demand first when an answer sounds too certain?
What you should see: choose the check that matches the claim: rows, witnesses, or transitions.

Tuesday individual activity

PRINCIPLE: MAKE YOUR OWN EVIDENCE

1. Complete the supplied E versus S truth table.
2. Give one satisfying assignment and explain why it is not a universal proof.
3. Trace valid, timeout, valid from Locked.
4. Name one condition the two-state model cannot establish.
5. Write one sentence separating what the evidence proves from what it leaves open.

What you should see: work individually first; if AI is used, record the exact row or transition you checked.

Preview: defend a bounded model

On Thursday, choose one path:

- Boolean/circuit: expression, circuit or simplification, and complete table; or
- state machine: states, inputs, transitions, start state, two traces, and a limit.

Defense norm

State the claim, show the row or trace, name the strongest limitation, answer a challenge, and critique the evidence rather than the person.

What you should see: a narrowed claim is a successful result when the challenge earns it.

Exit sentence

Complete this sentence

For this declared model, the evidence establishes _____; it does not establish _____ because _____.

- A truth table establishes behavior for the assignments it exhausts.
- A trace establishes the path for the inputs and transitions it names.
- A model limit protects the claim from becoming larger than its evidence.

One last check before trust

Claim	Minimum visible check
Expressions equivalent	every assignment in scope
Expression satisfiable	one named witness
Expression tautological	every assignment is true
Trace is correct	start state, inputs, and transitions
Model is complete	every declared state/input pair is accounted for

What you should see: the check is part of the claim, not an optional decoration.

Index 1 of 3: principle, slide, prompt

Slide	Title	Principle	Prompt
2	The question for today	MODEL SCOPE	
3	Our checking rhythm	PREDICT–RUN–EXPLAIN	
5	The tempting simplification	CHECK THE CLAIM	
6	Four assignments, four rows	EXHAUST SMALL CASES	
7	One counterexample is enough	COUNTEREXAMPLE	
8	Prompt 01: make the claim checkable	DEFINE BEFORE COMPUTING	01a
9	Prompt 02: make every row visible	EXHAUST SMALL CASES	02a
10	A correct simplification from the activity	COMPARE CLAIMS	
11	From expression to circuit	REPRESENTATION	

Index 2 of 3: principle, slide, prompt

Slide	Title	Principle	Prompt
12	Equivalence is a universal statement	MATCH THE SCOPE	
13	Satisfiable is not always true	SEPARATE CLAIMS	
14	Three small classifications	CHECK THE WORDING	
15	Prompt 03a: return counterexamples	TEST THE UNIVERSAL CLAIM	03a
16	Prompt 03b: witness versus tautology	DO NOT OVERCLAIM	03b
17	Truth tables and circuits agree when the representation agrees	CHECK THE REPRESENTATION	
18	Now change the question: how does it move?	FROM VALUES TO STATES	
19	Synthetic badge model	MAKE TRANSITIONS EXPLICIT	
20	Trace it one input at a time	CHECK THE TRACE	

Index 3 of 3: principle, slide, prompt

Slide	Title	Principle	Prompt
21	Prompt 04: run the complete model	REPRODUCIBLE TRACE	04a
22	A model can be incomplete	FIND WHAT WAS FORGOTTEN	
23	Prompt 05: name the missing transition	VALIDATE THE MODEL	05a
24	What the badge model cannot establish	MODEL LIMIT	
25	The three AI failure modes	FAILURE MODES	
26	Tuesday individual activity	MAKE YOUR OWN EVIDENCE	
27	Preview: defend a bounded model	EVIDENCE BEFORE CONFIDENCE	
28	Exit sentence	LIMIT THE CLAIM	
29	One last check before trust	CHECK THE WORK	